Comparative Analysis of Artificial Neural Networks and Convolutional Neural Networks

Student ID: [P2943206]
Course: [Artificial Intelligence (MSC)]
Date: 5 November 2025

TABLE OF CONTENTS

1. INTRODUCTION 3
1.1 Background and Motivation 3
2. DATASET AND PREPROCESSING 3
2.1 Fashion-MNIST Dataset Description 3
2.2 Data Splitting Strategy 3
2.3 Normalization 4
3. METHODOLOGY 4
3.1 Development Environment 4
3.2 Evaluation Metrics 4
4. MODEL ARCHITECTURES 5
4.1 Artificial Neural Network (ANN) Architecture Design 5
4.2 Convolutional Neural Network (CNN) Architecture Design 6
5. TRAINING STRATEGY 7
5.1 Optimization Algorithm 7
5.2 Loss Function 7
5.3 Callbacks and Regularization 7
5.3.1 Early Stopping 7
5.3.2 Learning Rate Reduction 7
5.4 Data Augmentation (CNN Only) 7
5.5 Training Configuration 8
6. RESULTS AND EVALUATION 8
6.1 Overall Performance 8
6.2 Training Dynamics 9
6.2.1 ANN Training 9
6.2.2 CNN Training 9
6.3 Per-Class Performance Analysis 9
6.4 Confusion Matrix Analysis 9
6.4.1 Top 3 Most Confused Class Pairs (ANN): 9
6.4.2 Top 3 Most Confused Class Pairs (CNN): 9
7. CRITICAL ANALYSIS 9
7.1 Why CNN Outperforms ANN 9
7.2 Error Analysis 10
7.2.1 Systematically Confused Classes: 10
7.3 Limitations 10
7.3.1 Dataset Limitation 10
7.3.2 Model Limitations 10
8.0 Reflection and improvements 10
8.1 Data Augmentation 10
8.2 Regularization, Dropout and Learning Rate 11
8.3 Architectural Enhancements 11
References 12
APPENDICES-1 13

1. INTRODUCTION

1.1 Background and Motivation

Image classification is one of the most important and result oriented tasks in the deep learning and computer vision field from tasks like disease classification to autonomous vehicles. There is a dataset named Fashion-MNIST that will be used in this project.

2. DATASET AND PREPROCESSING

2.1 Fashion-MNIST Dataset Description

In this Image classification task selected a dataset named Fashion-MNIST having a total 70,000 images.

```
Class counts with names in training data:
T-shirt/top    6000
Trouser        6000
Pullover       6000
Dress          6000
Coat           6000
Sandal         6000
Shirt          6000
Sneaker        6000
Bag            6000
Ankle boot     6000
Name: count, dtype: int64
```

Figure 1: Dataset classes and data distribution

dataset

In this dataset there are 10 distinct classes and uniform data 6000 images in each are distributed among these ten classes. The dimensions of the image are: 28×28×1 (grayscale) with pixel value range [0, 255] (uint8). The dataset demonstrates that each class has perfect dataset balance which is essential for model training and testing outcome.

2.2 Data Splitting Strategy

```
DATA SPLIT
Training set: 48000 samples (stratified)
Validation set: 12000 samples (stratified)
Test set: 10000 samples
```

Figure 2: Data split

The dataset is split 48,000 images (80%) for training, 12,000 images (20%) for validation step and 10,000 images for test (separate hold-out set). In this project applied stratified sampling technique which ensures that throughout training and test phase the dataset will be uniformly balanced among all ten classes.

2.3 Normalization

This project employed normalisation on images using min-max scaling technique so that pixel values would normalise and range from [0, 255] to [0, 1] range. This technique is crucial during model training because it handles inconsistent pixel intensity ranges, scales the data to a fixed range and provides faster model convergence. Ultimately helps in achieving better model performance during training and provides a generalized model (Schwarzhans *et al.*, 2025).

3. METHODOLOGY

This part will contain programming language, technologies, training method, hyper parameters, model evaluation metrics and deployment techniques to construct the solution.

3.1 Development Environment

For developing this project there are some hardware and software specifications required.

Table 1: Specifications for the project

IDE	Google colab
Framework	TensorFlow 2.19.0 with Keras API
Hardware	NVIDIA Tesla T4 GPU
Programming Language	Python 3.12
Key Libraries	NumPy, Matplotlib, Seaborn, Scikit-learn, Pandas

3.2 Evaluation Metrics

After model training, it is necessary to assess the model's mode performance. To do this, a number of classification metrics are used, including: Test accuracy is used to assess how well a model performs after it has been trained on a dataset that the model has never seen. A confusion matrix is a table that compares the predicted labels for each class to the ground

truth labels means per class accuracy. Another classification statistic is the Classification Report, which projects the performance of each class across models using metrics including recall, accuracy, and F1-Score in an image classification task (Fresz, Loercher and Huber, 2024). The precision metric calculates the percentage of times the model correctly predicted the actual occurrence or proportion of correct positive predictions. Recall tells that out of all positive instances how many were actually truly detected. F1-score metrics is a harmonic mean of precision and recall also it balances both as well as it is best suitable for measuring class-imbalanced dataset. F1Score= 2×Precision+RecallPrecision×Recall (GeeksforGeeks, 2025). Also training and validation loss metrics. (Viering and Loog, 2021).

4. MODEL ARCHITECTURES

4.1 Artificial Neural Network (ANN) Architecture Design

```
# Prepare data for ANN (flatten images)
X_train_flat = X_train.reshape((X_train.shape[0], -1))
X_val_flat = X_val.reshape((X_val.shape[0], -1))
X_test_flat = X_test.reshape((X_test.shape[0], -1))

print(f"Flattened training shape: {X_train_flat.shape}")

# Build ANN architecture
ann_model = models.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(512, activation='relu', name='dense_1'),
    layers.Dropout(0.3, name='dropout_1'),
    layers.Dense(256, activation='relu', name='dense_2'),
    layers.Dropout(0.3, name='dropout_2'),
    layers.Dense(128, activation='relu', name='dense_3'),
    layers.Dropout(0.2, name='dropout_3'),
    layers.Dense(10, activation='softmax', name='output')
], name='ANN_Model')

ann_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

ann_model.summary()
```

Figure 3: ANN architecture

Developed ANN architecture having overall 3 hidden layers. Architectural designs of this are flattened Input layers to 784-dimensional vectors (28 x 28) of 1-D array. The first hidden layer has a Dense Layer with Fully connected layer 512 neurons, ReLU activation so that it can understand complex patterns and Dropout_1 regularization layer drops 30% neurons randomly so that it can mitigate the overfitting problem. The second hidden layer Dense_2 consists of Fully connected layer 256 neurons, ReLU activation so that more abstract features can be learned and also has Dropout_2 regularization layer dropping 30% neurons. The third hidden layer Dense_3 consists of Fully connected layer 128 neurons, ReLU activation so that even fine-grained details can get extracted and also has Dropout_2 regularization layer dropping 30% neurons. The output layer consists of Fully connected layer,10 neurons, Softmax activation function output from (0-9) classes (GeeksforGeeks, 2025).

4.2 Convolutional Neural Network (CNN) Architecture Design

```python
print("5. BUILDING CNN MODEL (OPTIMIZED)")

# Prepare data for CNN (add channel dimension)
X_train_cnn = X_train[..., np.newaxis]
X_val_cnn = X_val[..., np.newaxis]
X_test_cnn = X_test[..., np.newaxis]

print(f"CNN training shape: {X_train_cnn.shape}")

# CNN Architecture
cnn_model = models.Sequential([
    layers.Input(shape=(28, 28, 1)),

    # Block 1
    layers.Conv2D(32, (3, 3), activation='relu', padding='same', name='conv2d_1'),
    layers.BatchNormalization(name='bn_1'),
    layers.Conv2D(32, (3, 3), activation='relu', padding='same', name='conv2d_2'),
    layers.BatchNormalization(name='bn_2'),
    layers.MaxPooling2D((2, 2), name='maxpool_1'),
    layers.Dropout(0.25, name='dropout_1'),

    # Block 2
    layers.Conv2D(64, (3, 3), activation='relu', padding='same', name='conv2d_3'),
    layers.BatchNormalization(name='bn_3'),
    layers.Conv2D(64, (3, 3), activation='relu', padding='same', name='conv2d_4'),
    layers.BatchNormalization(name='bn_4'),
    layers.MaxPooling2D((2, 2), name='maxpool_2'),
    layers.Dropout(0.25, name='dropout_2'),

    # Classifier
    layers.Flatten(name='flatten'),
    layers.Dense(256, activation='relu', name='dense_1'),
    layers.BatchNormalization(name='bn_5'),
    layers.Dropout(0.4, name='dropout_3'),
    layers.Dense(10, activation='softmax', name='output')
], name='CNN_Model')

cnn_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

cnn_model.summary()
```

Figure 4: CNN architecture

Overall 3 blocks and 17 layers in this CNN architecture.

In this CNN architecture first step is image size resized from 28*28 to (28,28,1). Block-1 does basic feature extraction tasks and in this Block-1 there are 2 Conv2D layers (32 filters) which learns edges, patterns and lines shapes from an image. In each 2 Conv2D layers (32 filters) there are separate BatchNormalization layers which stabilises the model. There is a MaxPooling2D(2×2) layer in Block-1 which shrinks the image size and lastly there is a Dropout(0.25) layer which removes 25% neurons which helps from overfitting of model.

Block-2 does advanced feature extraction tasks and has 2- Conv2D layers (64 filters) which learns a more complex pattern than the first block-1, 2 Conv2D layers. There are the same 2 separate BatchNormalization layers, MaxPooling2D(2×2), Dropout(0.25) in this block-2.

The picture is flattened from a 2-D to a 1-D list in the first layer by the Classifier block. The second Dense(256) layer adds all features and makes a final pattern. The third layer is

BatchNormalization. Fourth layer Dropout(0.4). Fifth and last layer Dense(10, softmax) provides prediction by model from (0-9) (Chen *et al.*, 2023).

5. TRAINING STRATEGY

5.1 Optimization Algorithm

In this project both ANN and CNN model used the same **Adam optimizer** with learning rate 0.001. The benefit of using this optimizer is it is a combination of two optimisers (RMSprop and momentum) Adam has excellent capabilities adjusting learning rate per epoch during training also minimising the risk of over-fitting.

5.2 Loss Function

This project implemented **Sparse Categorical Crossentropy** for multi-class classification because output have (0-9) labels.

5.3 Callbacks and Regularization

5.3.1 **Early Stopping**

In the ANN model implemented early stopping with patience of 5 epochs and for CNN model there is a patience of 10 epochs (longer due to data augmentation reason). Early stopping benefits models from over-fitting problems by stopping training when validation loss does not improve over defined epochs.

5.3.2 Learning Rate Reduction

In this project, the learning rate was reduced in both CNN and ANN models. This learning rate decreases by half as validation loss levels off and assists in the fine-tune later in the epochs. The learning rate of 1e-6 is the minimum rate to ensure that it is not overreducing.

5.4 Data Augmentation (CNN Only)

In this project implemented real-time data augmentation especially during CNN training.

```python
### Data Augmentation (ONLY for training!)
print("CREATING DATA AUGMENTATION (Training Only)")

datagen = ImageDataGenerator(
    rotation_range=10,
    width_shift_range=0.1,
    height_shift_range=0.1,
    zoom_range=0.1
)

datagen.fit(X_train_cnn)

print(" Data augmentation configured:")
print("  - Rotation: ±10°")
print("  - Width/Height shift: ±10%")
print("  - Zoom: ±10%")
print("  - Applied ONLY to training data")
```

```
CREATING DATA AUGMENTATION (Training Only)
 Data augmentation configured:
  - Rotation: ±10°
  - Width/Height shift: ±10%
  - Zoom: ±10%
  - Applied ONLY to training data
```

Figure 5: Data augmentation in CNN

5.5 Training Configuration

Table 2: training specs

Parameter	Value	Rationale
Batch Size	128	Balance between convergence speed and memory
Max Epochs	30 (ANN), 50 (CNN)	Sufficient for convergence with early stopping
Steps per Epoch	375 (48000/128)	Complete dataset coverage per epoch

6. RESULTS AND EVALUATION

6.1 Overall Performance

Refer to Appendices-1.

Table 3: CNN and ANN model performance

Model	**Test Accuracy**	**Test Loss**	**Validation Accuracy**	**Parameters**
ANN	88.98%	0.31	90.17%	567,434
CNN	92.67%	0.19	93.52%	872,426
Improvement	+4.19%	-12%	+3.35%	+53.7%

The key analysis from both model performance is that CNN model outperforms the ANN model in terms of performance and accuracy by +4.19%. In this experiment there is a lower

test loss in CNN compared to ANN which demonstrates CNN capabilities in accurate prediction of labels. Both CNN and ANN models have no over fitting which can be evaluated by demonstrating test accuracy is close to validation accuracy.

6.2 Training Dynamics

6.2.1 ANN Training

The ANN training model converged during the 27th epoch and highest validation accuracy measured at 22nd epoch(90.17%). Throughout training model performance was established and it showed improvement. At epoch(15,20) the learning rate reduced during ANN model training.

6.2.2 CNN Training

The CNN training model converged during the 48th epoch, meaning the model stopped training and highest validation accuracy measured at 38 epoch(93.52%). Throughout training model performance was established and it showed improvement. At epoch(24, 30, 42) the learning rate reduced during CNN model training.

6.3 Per-Class Performance Analysis

Refer appendices-1 for class wise performance.

Per class analysis provides critical analysis such as: shirt class showed most improvement from ANN to CNN model it is improved by 16.7%.

6.4 Confusion Matrix Analysis

6.4.1 Top 3 Most Confused Class Pairs (ANN):

In ANN top 3 most confused in a number of classes, including: Shirt- T-shirt/top: 145 samples.Pullover-coat: 104 samples, Coat-Pullover: 93 samples.

6.4.2 Top 3 Most Confused Class Pairs (CNN):

Top 3 Most confused in a number of classes in CNN, including: T-shirt/top -Shirt: 145 samples, Pullover- Shirt: 76 samples, Coat-Shirt: 67 samples.

7. CRITICAL ANALYSIS

7.1 Why CNN Outperforms ANN

One of the most important reasons of CNN outperforms ANN is spatial features holding capability because CNN use local receptive fields, reserving important and fine-grained features from an image even shapes, sizes and edges while ANN flatten the image from

28*28 to direct 784 pixels also it loses the spatial features due to each pixel acting as separate features.

Another reason is CNN's parameters apply throughout all spatial locations despite having only 228 parameters from a 3x3 filter with 32 channels. On the other hand,

Another reason is Hierarchical Feature Learning. In CNN there is a fixed pattern of learning features in CNN convolutional layers: edges-textures-parts-objects. Whereas, ANN does not have this hierarchy based learning features instead, in ANN all features learned at once.

7.2 Error Analysis

7.2.1 Systematically Confused Classes:

Shirt vs T-shirt/top systematically is the most confused class and both have upper body garment. Maybe confusion is due to similar patterns in neck design detail, and garment shape. Another reason to be low resolution (28×28) limits it can impact the model classification capability between these two.

Another most confused class is Pullover vs Coat may be due to both having long sleeve outwear. Maybe fine details such as knit vs smooth not properly get captured due to low resolutions.

7.3 Limitations

7.3.1 Dataset Limitation

The input dataset is not sufficient for model training because of low resolutions and fine-grain details not getting captured. Sometimes gray-scale information eliminates the color identification(blue jeans vs black trousers).

7.3.2 Model Limitations

To capture exact correct details from discriminative regions there is a need for attention mechanisms in model architecture. The 3-layer CNN (also known as shallow CNN) can be underutilizing representational capacity.

8.0 Reflection and improvements

8.1 Data Augmentation

Data augmentation used in CNN architecture especially in training data which assists in increasing more datasets and mitigating overfitting/ underfitting. Increasing the dataset in numbers through data augmentation helps models identify complexity capability from the dataset. But still this method is very traditional. Advanced data augmentation techniques

such as cutout that is used for erasing robustness from the model and enhances the model robustness. Another MixUp advanced technique can add two images together helps in improved model object understanding and makes the model generalized (Kumar *et al.*, 2023).

8.2 Regularization, Dropout and Learning Rate

In this project both ANN and CNN model used the same **Adam optimizer** with learning rate 0.001 and using dropout dropped neurons till max 40%. These all three techniques used so that the model could prevent overfitting. By adopting some good model parameters mode can be better such as cosine annealing (Janson *et al.*, 2025). Similarly if the dataset is small and architecture can lead to underfitting instead of drop neurons one can use advanced regularisation methods such as DropConnect, Stochastic Depth can prevent from model overfitting (Salehin and Kang, 2023).

8.3 Architectural Enhancements

This simple CNN 2-D convolution and simple 3 block based CNN model is not enough to get a good generalized model. By adopting a better CNN based model or doing changes inside model architecture needed. Instead of building a model from scratch we can adopt already pre-trained CNN based model such as: ResNet-50, ResNet-101, or ResNet-152 which can have advanced capabilities such as: can handle vanishing gradient problem, have ResNet blocks in the model, model that have excellent capabilities in focusing on discriminative regions, make attention to tiny fine grained details, global structure. A model that can reduce spatial dimensions through pooling layers. (Liu and Goh, 2025).

References

Chen, Z., Lv, M., Tian, S. and Yin, S. (2023) ‘Fashion-MNIST classification based on CNN image recognition’, *Highlights in Science, Engineering and Technology*, 34, pp. 196–202. doi: 10.54097/hset.v34i.5463. Available under CC BY-NC 4.0 licence.

Fresz, B., Loercher, L. and Huber, M.F. (2024) ‘Classification Metrics for Image Explanations: Towards Building Reliable XAI-Evaluations’, *arXiv preprint* arXiv:2406.05068v1 [cs.CV], 7 June. Available at: https://arxiv.org/abs/2406.05068

GeeksforGeeks (2025) *F1 Score in Machine Learning*. Available at: https://www.geeksforgeeks.org/machine-learning/f1-score-in-machine-learning/ (Last updated: 23 July 2025) (Accessed: [23 november 2025]).

GeeksforGeeks (2025) *Artificial Neural Networks and its Applications*. Last updated: 12 July. Available at: https://www.geeksforgeeks.org/artificial-intelligence/artificial-neural-networks-and-its-applications/ (Accessed: [November-2025]).

Janson, P., Singh, V., Mehrbod, P., Ibrahim, A., Rish, I., Belilovsky, E. and Thérien, B. (2025) ‘Beyond Cosine Decay: On the effectiveness of Infinite Learning Rate Schedule for Continual Pre-training’, *arXiv preprint* arXiv:2503.02844v1 [cs.LG], 4 March. Available at: https://arxiv.org/abs/2503.02844

Kumar, T., Mileo, A., Brennan, R. and Bendechache, M. (2023) ‘Image Data Augmentation Approaches: A Comprehensive Survey and Future Directions’, *arXiv preprint* arXiv:2301.02830v4 [cs.CV], 12 March. Available at: https://arxiv.org/abs/2301.02830

Liu, X. and Goh, K.M. (2025) ‘ResNet: Enabling Deep Convolutional Neural Networks through Residual Learning’, *arXiv preprint* arXiv:2510.24036v1 [cs.CV], 28 October. Available at: https://arxiv.org/abs/2510.24036

Salehin, I. and Kang, D-K. (2023) ‘A review on dropout regularization approaches for deep neural networks within the scholarly domain’, *Electronics*, 12(14), p. 3106. doi: 10.3390/electronics12143106.

Schwarzhans, F., George, G., Escudero Sanchez, L., Abraham, J.E., Woitek, R. and Hatamikia, S. (2025) ‘Image normalization techniques and their effect on the robustness and predictive power of breast MRI radiomics’, *European Journal of Radiology*, 187, p. 112086. doi: 10.1016/j.ejrad.2025.112086.

Viering, T. and Loog, M. (2021) ‘The Shape of Learning Curves: A Review’, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, [online]. Delft University of Technology, the Netherlands; University of Copenhagen, Denmark.

APPENDICES-1

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```
print(f"Training data shape: {X_train_full.shape}")
print(f"Training labels shape: {y_train_full.shape}")
print(f"Test data shape: {X_test.shape}")
print(f"Test labels shape: {y_test.shape}")
print(f"Number of classes: {len(class_names)}")
print(f"Pixel value range: [{X_train_full.min()}, {X_train_full.max()}]")
```

```
Training data shape: (60000, 28, 28)
Training labels shape: (60000,)
Test data shape: (10000, 28, 28)
Test labels shape: (10000,)
Number of classes: 10
Pixel value range: [0, 255]
```

Figure - 1

```
# Map class indices to class names
class_counts.index = [class_names[i] for i in class_counts.index]
print("\nClass counts with names in training data:")
print(class_counts)
```

```
Class counts with names in training data:
T-shirt/top    6000
Trouser        6000
Pullover       6000
Dress          6000
Coat           6000
Sandal         6000
Shirt          6000
Sneaker        6000
Bag            6000
Ankle boot     6000
Name: count, dtype: int64
```

Figure -2

```
print("DATA SPLIT")
print(f"Training set: {X_train.shape[0]} samples (stratified)")
print(f"Validation set: {X_val.shape[0]} samples (stratified)")
print(f"Test set: {X_test.shape[0]} samples")
```

```
DATA SPLIT
Training set: 48000 samples (stratified)
Validation set: 12000 samples (stratified)
Test set: 10000 samples
```

```
# Normalizing pixel values to [0, 1]
X_train = X_train.astype('float32') / 255.0
X_val = X_val.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0

print("Normalization complete: Pixel values scaled to [0, 1]")
```

```
Normalization complete: Pixel values scaled to [0, 1]
```

Figure - 3

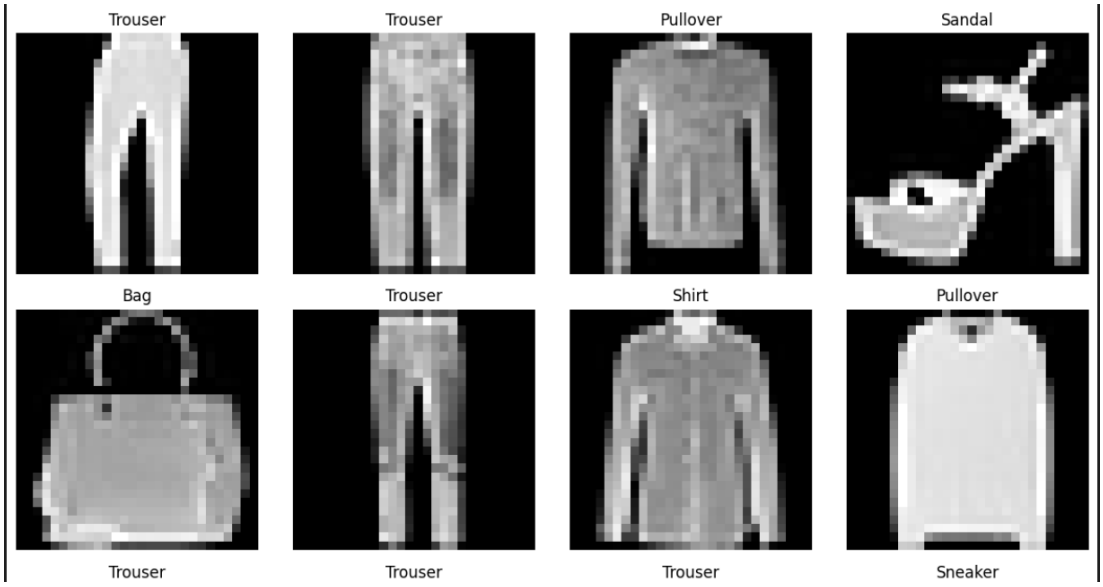

Figure - 4

```python
print("4. BUILDING ANN MODEL")

# Prepare data for ANN (flatten images)
X_train_flat = X_train.reshape((X_train.shape[0], -1))
X_val_flat = X_val.reshape((X_val.shape[0], -1))
X_test_flat = X_test.reshape((X_test.shape[0], -1))

print(f"Flattened training shape: {X_train_flat.shape}")

# Build ANN architecture
ann_model = models.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(512, activation='relu', name='dense_1'),
    layers.Dropout(0.3, name='dropout_1'),
    layers.Dense(256, activation='relu', name='dense_2'),
    layers.Dropout(0.3, name='dropout_2'),
    layers.Dense(128, activation='relu', name='dense_3'),
    layers.Dropout(0.2, name='dropout_3'),
    layers.Dense(10, activation='softmax', name='output')
], name='ANN_Model')

ann_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

ann_model.summary()
```

Figure 5 - ANN model/ architecture

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 512)	401,920
dropout_1 (Dropout)	(None, 512)	0
dense_2 (Dense)	(None, 256)	131,328
dropout_2 (Dropout)	(None, 256)	0
dense_3 (Dense)	(None, 128)	32,896
dropout_3 (Dropout)	(None, 128)	0
output (Dense)	(None, 10)	1,290

Total params: 567,434 (2.16 MB)

Trainable params: 567,434 (2.16 MB)

Non-trainable params: 0 (0.00 B)

Figure 6 - ANN model parameters

```python
# CNN Architecture
cnn_model = models.Sequential([
    layers.Input(shape=(28, 28, 1)),

    # Block 1
    layers.Conv2D(32, (3, 3), activation='relu', padding='same', name='conv2d_1'),
    layers.BatchNormalization(name='bn_1'),
    layers.Conv2D(32, (3, 3), activation='relu', padding='same', name='conv2d_2'),
    layers.BatchNormalization(name='bn_2'),
    layers.MaxPooling2D((2, 2), name='maxpool_1'),
    layers.Dropout(0.25, name='dropout_1'),

    # Block 2
    layers.Conv2D(64, (3, 3), activation='relu', padding='same', name='conv2d_3'),
    layers.BatchNormalization(name='bn_3'),
    layers.Conv2D(64, (3, 3), activation='relu', padding='same', name='conv2d_4'),
    layers.BatchNormalization(name='bn_4'),
    layers.MaxPooling2D((2, 2), name='maxpool_2'),
    layers.Dropout(0.25, name='dropout_2'),

    # Classifier
    layers.Flatten(name='flatten'),
    layers.Dense(256, activation='relu', name='dense_1'),
    layers.BatchNormalization(name='bn_5'),
    layers.Dropout(0.4, name='dropout_3'),
    layers.Dense(10, activation='softmax', name='output')
], name='CNN_Model')

cnn_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

Figure 7 - CNN model

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 28, 28, 32)	320
bn_1 (BatchNormalization)	(None, 28, 28, 32)	128
conv2d_2 (Conv2D)	(None, 28, 28, 32)	9,248
bn_2 (BatchNormalization)	(None, 28, 28, 32)	128
maxpool_1 (MaxPooling2D)	(None, 14, 14, 32)	0
dropout_1 (Dropout)	(None, 14, 14, 32)	0
conv2d_3 (Conv2D)	(None, 14, 14, 64)	18,496
bn_3 (BatchNormalization)	(None, 14, 14, 64)	256
conv2d_4 (Conv2D)	(None, 14, 14, 64)	36,928
bn_4 (BatchNormalization)	(None, 14, 14, 64)	256
maxpool_2 (MaxPooling2D)	(None, 7, 7, 64)	0
dropout_2 (Dropout)	(None, 7, 7, 64)	0
flatten (Flatten)	(None, 3136)	0
dense_1 (Dense)	(None, 256)	803,072
bn_5 (BatchNormalization)	(None, 256)	1,024
dropout_3 (Dropout)	(None, 256)	0
output (Dense)	(None, 10)	2,570

Total params: 872,426 (3.33 MB)

Trainable params: 871,530 (3.32 MB)

Non-trainable params: 896 (3.50 KB)

Figure 7 - CNN model parameters

6. MODEL EVALUATION

```python
print("6. MODEL EVALUATION")

# Evaluate ANN
ann_test_loss, ann_test_acc = ann_model.evaluate(X_test_flat, y_test, verbose=0)
print(f"\n ANN Test Accuracy: {ann_test_acc*100:.2f}%")
print(f" ANN Test Loss: {ann_test_loss:.4f}")
```

```
6. MODEL EVALUATION

 ANN Test Accuracy: 88.98%
 ANN Test Loss: 0.3168
```

```python
# Evaluate CNN
cnn_test_loss, cnn_test_acc = cnn_model.evaluate(X_test_cnn, y_test, verbose=0)
print(f"\n CNN Test Accuracy: {cnn_test_acc*100:.2f}%")
print(f" CNN Test Loss: {cnn_test_loss:.4f}")
```

```
 CNN Test Accuracy: 92.67%
 CNN Test Loss: 0.1979
```

Figure 8 - model test accuracy

```
ANN - Classification Report:
              precision    recall  f1-score   support

 T-shirt/top       0.82      0.87      0.85      1000
     Trouser       0.98      0.98      0.98      1000
    Pullover       0.81      0.81      0.81      1000
       Dress       0.90      0.89      0.89      1000
        Coat       0.77      0.84      0.80      1000
      Sandal       0.98      0.97      0.97      1000
       Shirt       0.75      0.66      0.70      1000
     Sneaker       0.94      0.96      0.95      1000
         Bag       0.98      0.97      0.98      1000
  Ankle boot       0.96      0.95      0.96      1000

    accuracy                           0.89     10000
   macro avg       0.89      0.89      0.89     10000
weighted avg       0.89      0.89      0.89     10000


ANN - Per-Class Accuracy:
T-shirt/top    : 87.20%
Trouser        : 97.60%
Pullover       : 80.80%
Dress          : 88.70%
Coat           : 83.90%
Sandal         : 96.80%
Shirt          : 65.80%
Sneaker        : 96.20%
```

Figure -9

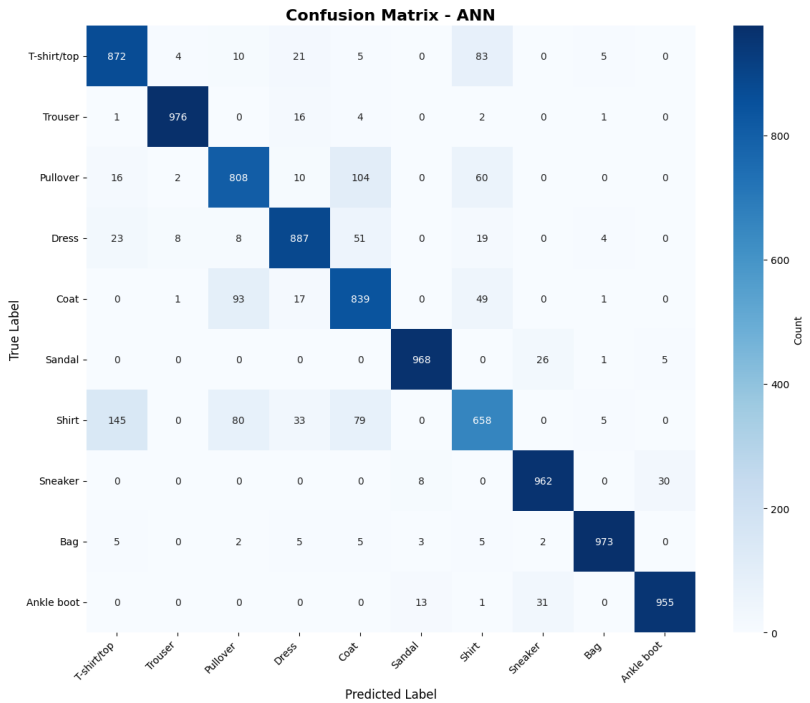

Figure - 10

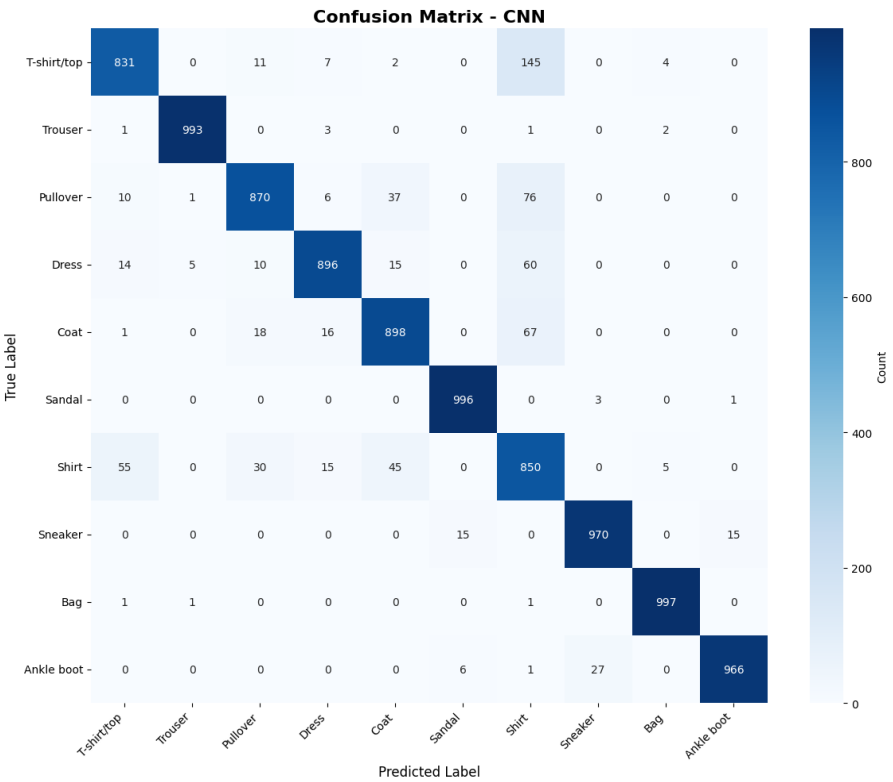

Figure - 11

```
CNN - Classification Report:
              precision    recall  f1-score   support

 T-shirt/top       0.91      0.83      0.87      1000
     Trouser       0.99      0.99      0.99      1000
    Pullover       0.93      0.87      0.90      1000
       Dress       0.95      0.90      0.92      1000
        Coat       0.90      0.90      0.90      1000
      Sandal       0.98      1.00      0.99      1000
       Shirt       0.71      0.85      0.77      1000
     Sneaker       0.97      0.97      0.97      1000
         Bag       0.99      1.00      0.99      1000
  Ankle boot       0.98      0.97      0.97      1000

    accuracy                           0.93     10000
   macro avg       0.93      0.93      0.93     10000
weighted avg       0.93      0.93      0.93     10000


CNN - Per-Class Accuracy:
T-shirt/top     : 83.10%
Trouser         : 99.30%
Pullover        : 87.00%
Dress           : 89.60%
Coat            : 89.80%
Sandal          : 99.60%
Shirt           : 85.00%
Sneaker         : 97.00%
Bag             : 99.70%
Ankle boot      : 96.60%
```

Figure - 12

```
PER-CLASS ACCURACY IMPROVEMENT (CNN vs ANN)
      Class ANN Accuracy CNN Accuracy Improvement
T-shirt/top       87.20%       83.10%      -4.10%
    Trouser       97.60%       99.30%      +1.70%
   Pullover       80.80%       87.00%      +6.20%
      Dress       88.70%       89.60%      +0.90%
       Coat       83.90%       89.80%      +5.90%
     Sandal       96.80%       99.60%      +2.80%
      Shirt       65.80%       85.00%     +19.20%
    Sneaker       96.20%       97.00%      +0.80%
        Bag       97.30%       99.70%      +2.40%
 Ankle boot       95.50%       96.60%      +1.10%

Biggest improvement: Shirt (19.20%)
Only class worse: T-shirt/top (-4.10%)
```

Figure - 13

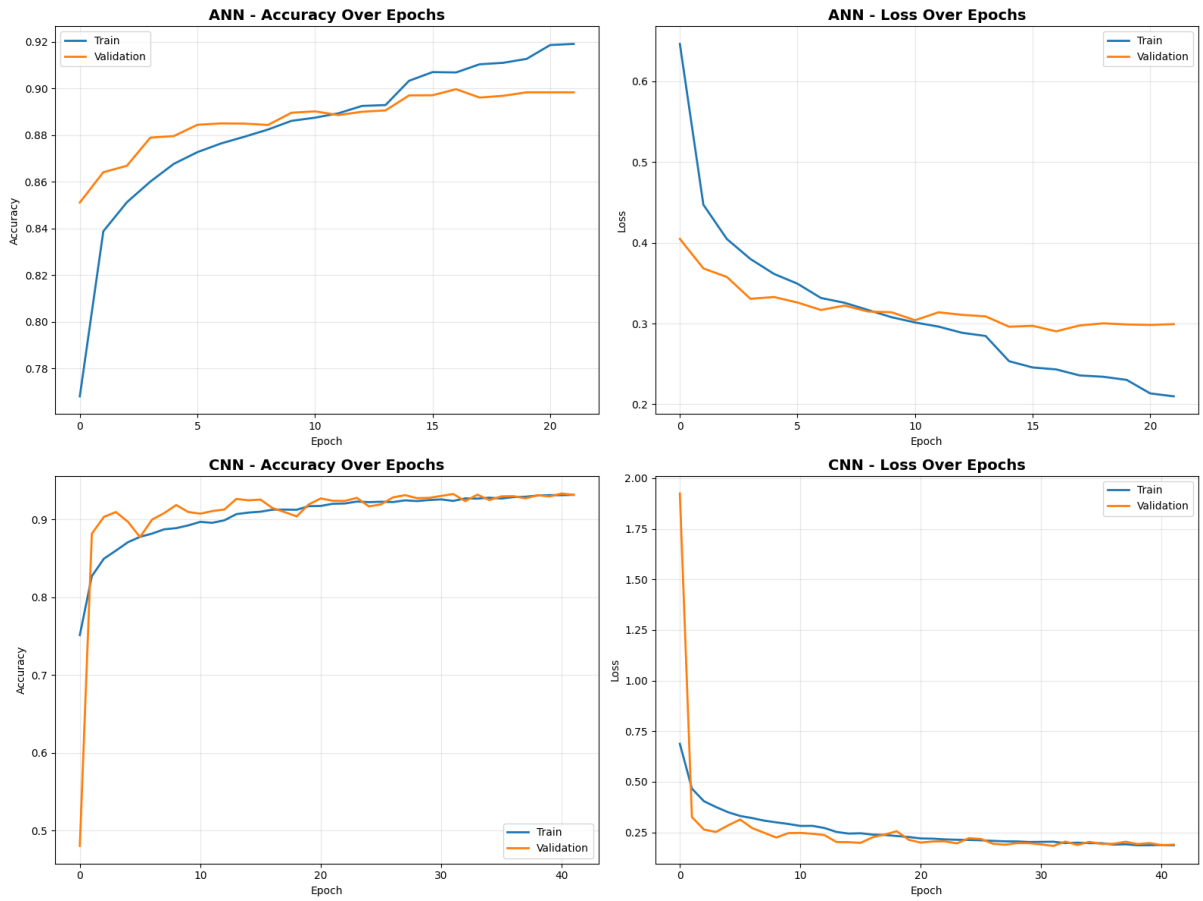

Figure - 14

```
9. ERROR ANALYSIS - CONFUSED CLASSES

ANN - Most Confused Class Pairs:
Shirt           → T-shirt/top    :  145 samples
Pullover        → Coat           :  104 samples
Coat            → Pullover       :   93 samples
T-shirt/top     → Shirt          :   83 samples
Shirt           → Pullover       :   80 samples
Shirt           → Coat           :   79 samples
Pullover        → Shirt          :   60 samples
Dress           → Coat           :   51 samples
Coat            → Shirt          :   49 samples
Shirt           → Dress          :   33 samples

CNN - Most Confused Class Pairs:
T-shirt/top     → Shirt          :  145 samples
Pullover        → Shirt          :   76 samples
Coat            → Shirt          :   67 samples
Dress           → Shirt          :   60 samples
Shirt           → T-shirt/top    :   55 samples
Shirt           → Coat           :   45 samples
Pullover        → Coat           :   37 samples
Shirt           → Pullover       :   30 samples
Ankle boot      → Sneaker        :   27 samples
Coat            → Pullover       :   18 samples
```

Figure - 15